# Portable AI-Assisted Software Support

A Proposal for a Standardized, Self-Contained Documentation Framework

Protocol ID: `MMG-PASD-1.0`

**Status:** *Proposed Specification — Public Draft*

Djeff Bee*

*Principal Architect, Meaningfulness Media Group*

October 25, 2026

**Abstract**

Traditional software documentation optimizes for human searchability, requiring users to translate observable symptoms into developer terminology, navigate fragmented pages, and synthesize recovery procedures manually. While general-purpose Large Language Models (LLMs) can perform this synthesis interactively, querying an ungrounded assistant about specific software routinely produces hallucinated settings and unsafe diagnostic advice. This paper proposes the *Portable AI Support Documentation* (PASD) specification: a vendor-neutral framework for distributing a complete, self-contained technical support knowledge base (`AI-SUPPORT.md`) alongside application source code. PASD separates normative document requirements (Layer A) from recommended generation workflows (Layer B) and empirical support-quality evaluation (Layer C). By formalizing a three-state epistemic model (*Supported*, *Explicitly Unsupported*, and *Unknown/Undocumented*), explicit third-party responsibility boundaries, and symptom-oriented diagnostic trees, PASD establishes a testable support contract. Operating under a "Bring Your Own AI" (BYOAI) model, the specification enables independent developers to provide grounded, self-service technical support without operating proprietary chatbot infrastructure.

**Keywords:** AI-Ready Documentation, Technical Support, LLM Grounding, BYOAI, Hallucination Mitigation, Software Documentation Standards.

**Reference Implementation & Specification Artifacts**

| | |
|---|---|
| **Reference Application:** | *MyFileExplorer* (`https://github.com/dobrado76/MyFileExplorer`) |
| **Support Artifact:** | Root-level `AI-SUPPORT.md` (Raw Markdown Payload) |
| **Extraction Specification:** | `AI-SUPPORT-documentation-generation-prompt.md` |
| **Maintenance Runbook:** | 5-Step Staged Generation & Review Protocol |

*Correspondence: info@meaningfulness.com.au

Copyright © 2026 Meaningfulness Media Group.
Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

# 1 Introduction: The Support Bottleneck

Software documentation has expanded substantially in scope and tooling over the past two decades. Projects routinely publish README files, wikis, searchable documentation portals, issue trackers, and reference manuals. Yet software engineers report that documentation is often outdated and that finding useful material can be difficult [Lethbridge et al., 2003], leaving the operational experience of obtaining support for complex software fragile.

When a user encounters an unexpected behavior, they face a terminological translation problem: they must map an observable symptom (e.g., "the preview pane is blank") to the internal vocabulary chosen by the developer (e.g., "syntax-highlighting renderer timeout"). They must then locate the relevant documentation pages, reconcile interactions between configuration settings and external dependencies, and determine a safe corrective action. When this manual synthesis fails, the user escalates the question to the maintainer.

For large commercial vendors with dedicated support staff, this model is manageable. For independent developers and small open-source or source-available projects, recurring questions about already-understood behavior represent a persistent maintenance burden.

At the same time, millions of users now have immediate access to capable Large Language Models (LLMs) that excel at natural-language symptom interpretation and multi-document synthesis. However, generative models can produce fluent text that is unfaithful to the supplied source [Ji et al., 2023]. Instruction-tuned assistants can also conform to a user's false premise [Sharma et al., 2024] and may fail to abstain when evidence is insufficient [Wen et al., 2025]. In software support, those general failure modes can manifest as invented menu paths, configuration keys, or command-line flags when a user asks how to configure a nonexistent feature.

This paper addresses a practical systems-engineering question: **how should a software project structure and distribute its operational knowledge so that a user's existing AI assistant can provide accurate, bounded first-line technical support?**

We propose the **Portable AI Support Documentation (PASD)** framework. Rather than requiring every developer to host a proprietary Retrieval-Augmented Generation (RAG) chatbot or pay for recurring API infrastructure, PASD specifies a single, self-contained, version-bounded Markdown document—canonically named `AI-SUPPORT.md`—distributed at the root of the project repository.

To maintain a clean separation of concerns, PASD is organized into three distinct layers:

1. **Layer A (Normative Document Specification):** The required structural, epistemic, and boundary properties of a conformant `AI-SUPPORT.md` file, independent of how the file is authored (Sections 4–6).

2. **Layer B (Recommended Maintenance Lifecycle):** A reproducible, staged workflow for generating, reviewing, bundling, and validating the documentation using AI coding assistants and Continuous Integration (Section 7).

3. **Layer C (Empirical Evaluation Protocol):** A falsifiable testing methodology for measuring whether the documentation improves support answer accuracy and safe abstention across language models (Section 8).

The guiding principle is straightforward: *the developer supplies the verified knowledge; the user supplies the AI assistant.*

# 2 Background and Prior Art

AI-consumable documentation is an active area of community experimentation and commercial tooling. PASD does not claim to invent machine-readable documentation; rather, it formalizes a specific, underserved profile within that ecosystem: a portable, vendor-neutral end-user support contract.

## 2.1 Discovery vs. Code Modification: `llms.txt` and `AGENTS.md`

Two widely discussed community conventions address adjacent problems. The `llms.txt` proposal [Howard, 2024] defines a root-level Markdown index designed to help web crawlers and AI systems discover important documentation URLs on a website, alongside an optional concatenated `llms-full.txt` file. While `llms-full.txt` shares the single-file delivery mechanism, `llms.txt` does not mandate symptom-oriented troubleshooting trees, explicit declarations of unsupported features, third-party responsibility boundaries, or support-evaluation test suites.

Conversely, `AGENTS.md` [OpenAI et al., 2025] provides repository-specific instructions for autonomous coding agents (such as build commands, test runners, and coding conventions). Its target persona is an agent modifying source code, whereas PASD targets an assistant helping an end user install, configure, operate, and troubleshoot the application without reading source code.

## 2.2 Structured AI Documentation vs. Support Contracts: AIRD

The AI-Ready Documentation (AIRD) specification (v1.2) [AIRD Community, 2026] proposes a five-layer architecture encompassing discovery via `llms.txt`, structural consistency, explicit AI-context metadata, feedback loops, and actionable execution blocks with verification and fallback steps. AIRD is the closest prior art to PASD.

However, AIRD focuses primarily on making technical and API documentation actionable for autonomous execution and developer workflows. PASD complements AIRD by narrowing the scope to *first-line end-user product support*, introducing mandatory three-state epistemic boundaries (explicitly documenting anti-features), third-party jurisdictional tables, and negative-case hallucination benchmarks.

### 2.3 Commercial RAG Support Platforms

Commercial documentation hosts and AI support services provide integrated chatbots backed by vector retrieval [Lewis et al., 2020, Gao et al., 2023]. These systems offer capabilities a static file cannot, including live telemetry, user-account integration, and analytics. However, they require hosted infrastructure, ongoing subscription or inference costs, and reliance on a specific provider. PASD targets projects that require zero-infrastructure, offline-capable, and vendor-neutral support distribution. Table 1 summarizes how PASD compares to existing approaches.

**Table 1:** Comparison of AI Documentation and Support Approaches

| Approach | Primary Audience | Delivery Mechanism | Distinguishing Scope & Limitations |
|---|---|---|---|
| **Conventional Docs / `README.md`** | Human readers | Multi-page web docs / repo files | Optimizes for human browsing; lacks explicit anti-hallucination boundaries for LLMs. |
| **`llms.txt` / `llms-full.txt`** | AI crawlers & search tools | Website root index / concatenated dump | Optimizes content discovery and ingestion; does not mandate troubleshooting schemas or negative space rules. |
| **`AGENTS.md`** | AI coding agents | Repository root file | Guides code modification, builds, and linting; not intended for end-user product support. |
| **AIRD (v1.2)** | AI agents & developers | Structured Markdown with metadata | Defines 5 layers for actionable execution and freshness; does not mandate end-user negative-case support testing. |
| **Hosted RAG Chatbots** | End users | Cloud SaaS widget / API | Provides live interactive support; requires hosting, recurring cost, and network connectivity. |
| **PASD (`AI-SUPPORT.md`)** | End users via BYOAI | Single portable root Markdown file | Mandates three-state capability boundaries, symptom triage, third-party demarcation, and negative-case evaluation. |

## 3 The Architectural Pivot: Bring Your Own AI (BYOAI)

The architectural premise of PASD is the separation of knowledge maintenance from inference execution. Instead of embedding an AI service into the application, the maintainer publishes a verified, structured knowledge artifact ($K$), and the user supplies their chosen language model ($M$) to interpret it.

## 3.1 A Conceptual Model of Grounded Support

To clarify what a support document can and cannot guarantee, we model a single-turn or multi-turn support interaction as a mapping from five inputs to a three-dimensional outcome tuple:

$$O = f(Q, K, V, E, M) = (C, S, D) \tag{1}$$

The inputs are:

- $Q$: The user's natural-language question or reported symptom.
- $K$: The supplied project knowledge base ($K = \emptyset$ when ungrounded; $K = K_{\text{PASD}}$ when using `AI-SUPPORT.md`).
- $V$: The target application version and operating environment.
- $E$: Diagnostic evidence supplied by the user (e.g., error messages, log excerpts, current settings).
- $M$: The user's selected language model and its inference parameters.

Crucially, we distinguish three separate outcome dimensions rather than conflating a good answer with a solved problem:

1. **Response Factual Correctness ($C \in [0, 1]$):** Whether the assistant's claims regarding application features, UI controls, configuration keys, and version behavior match the verified implementation.
2. **Diagnostic and Action Safety ($S \in [0, 1]$):** Whether the assistant's recommended checks are non-destructive, respect third-party responsibility boundaries, and appropriately abstain or request missing evidence $E$ when $Q$ is underspecified.
3. **Downstream Issue Resolution ($D \in \{0, 1\}$):** Whether executing the recommended procedure actually resolves the user's underlying operational problem in environment $V$.

This distinction is essential for scientific precision. An assistant that responds, *"I cannot determine why the export failed without seeing the error code in the log,"* achieves high factual correctness ($C$) and high diagnostic safety ($S$), even though downstream resolution ($D$) has not yet occurred. Furthermore, if the underlying issue is an unpatched software defect in version $V$, no documentation $K$ can yield $D = 1$; the optimal outcome is safe diagnosis and accurate bug-report escalation. Finally, $C$ and $S$ remain joint functions of both $K$ and $M$: improving $K$ reduces the need for model guessing, but retrieval and reasoning accuracy remain bounded by the capabilities and context window of the chosen model $M$.

### 3.2 Delivery Mechanisms: File Upload and Optional URL Routing

Because AI assistants vary in their ability to access external network resources, PASD defines two delivery tiers:

1. **Baseline Interoperable Delivery (Direct File Attachment):** The primary recommended interoperable delivery method for assistants supporting file attachments consists of downloading `AI-SUPPORT.md` from the repository (or locating it in the local application directory) and uploading or attaching it directly to the user's AI assistant. This method does not depend on web-browsing permissions, firewall rules, or crawler behavior.

2. **Optional Convenience Delivery (Raw URL and In-App Routing):** For assistants with verified web-fetching capabilities, applications MAY provide a direct link to the raw Markdown file (e.g., via `raw.githubusercontent.com`) or an in-app "Copy AI Support Link" button that copies the software version $V$, active error string $E$, and the raw URL to the clipboard. Because some assistants restrict URL fetching, truncate long web pages, or fail silently when a URL cannot be retrieved, embedded guidance SHOULD instruct the assistant to confirm that it has successfully read the document before answering.

### 3.3 Offline and Local-Model Portability

Because `AI-SUPPORT.md` is an ordinary UTF-8 Markdown file with no external runtime dependencies, it can be bundled directly into desktop installers and release archives. Users operating in air-gapped environments or using local inference engines (such as Ollama or LM Studio) can supply the file through clients that support attachment or document ingestion without transmitting telemetry or private file paths to third-party cloud providers. Where a client supports neither, the same Markdown can be pasted or incorporated through that client's local retrieval workflow.

## 4 Layer A: Normative Document Specification

Layer A defines the required structure and content of a PASD-compliant `AI-SUPPORT.md` artifact. Conformance to Layer A is a property of the published document itself; a project can achieve full Layer A compliance whether the file is written manually by human engineers or generated with AI assistance. The key words "MUST", "MUST NOT", "REQUIRED", "SHOULD", and "MAY" are to be interpreted as described in IETF RFC 2119 [Bradner, 1997].

## 4.1 Discoverability and Access

A conformant project MUST explicitly advertise the availability of AI-assisted support. The repository's root `README.md` MUST contain a prominent section near the top (e.g., "Need Help? Ask an AI Assistant") that links directly to the canonical `AI-SUPPORT.md` file and provides a concise procedure for uploading the file (or providing the raw link where supported) to an AI assistant.

## 4.2 Consolidation and Scope Constraint

The document MUST be self-contained for routine end-user support, incorporating the actual contents of relevant user guides, configuration references, troubleshooting procedures, and FAQs rather than mere summaries or external links. An AI assistant operating without repository access must be able to resolve ordinary user questions from this single file alone. Conversely, internal maintainer documentation (such as build-system internals, class architectures, or contribution rules) SHOULD be kept in separate files unless it directly aids in diagnosing user-observable behavior, preserving context capacity for support-relevant material.

## 4.3 Verified Factuality

Every technical claim within the document MUST be derived from the application's verified implementation. Because prose documentation commonly drifts behind implementation changes [Lethbridge et al., 2003], PASD adopts the normative rule that executable source code and automated test fixtures take precedence over legacy documentation, comments, or authorial intent. All documented UI labels, commands, paths, configuration keys, defaults, and error messages MUST match the actual application. Where a behavior cannot be verified with confidence, it MUST be explicitly identified as unverified rather than inferred.

## 4.4 Symptom-Oriented Troubleshooting and Data Safety

Users typically report problems in terms of observable symptoms rather than internal modules. The troubleshooting section MUST include a symptom index and organize diagnostic entries using the deterministic sequence:

$$\textit{Symptom} \rightarrow \textit{Possible Causes} \rightarrow \textit{Diagnostic Checks} \rightarrow \textit{Safe Resolution} \rightarrow \textit{Expected Result}$$

Procedures that modify or delete user data, reset configuration files, clear caches, overwrite state, or reinstall components MUST NOT be recommended as generic initial steps. Every destructive procedure MUST explicitly specify: (1) what data or state will be affected, (2) what could be lost, (3) what should be backed up beforehand, and (4) how to execute the operation safely.

### 4.5 The Codebase-Derived FAQ

The document MUST incorporate a comprehensive FAQ derived systematically from the application's actual user-facing controls, workflows, settings, file formats, integrations, and edge cases. It MUST NOT be populated with generic filler to inflate question counts. Questions SHOULD be phrased in natural language corresponding to how users actually ask for help (e.g., "Why did the application behave differently than expected?", "Where is this setting stored?", or "Can I undo this operation?"), and each answer SHOULD be self-contained and actionable.

### 4.6 Version Bounding

To prevent ambiguity when training data, repository material, or user context contains information from multiple releases, the document MUST state the exact application version, supported release range, or commit revision it describes in its header metadata. The embedded guidance MUST instruct assistants to clarify the user's version whenever troubleshooting version-sensitive behavior.

### 4.7 Testable Conformance Matrix

To allow maintainers and auditors to verify Layer A compliance objectively, Table 2 defines ten core conformance requirements and their verification methods.

## 5 The Three-State Epistemic Model and Negative Space Mandate

Conventional software manuals are almost exclusively positive: they describe features that exist. When an end user asks an LLM how to perform an action that the software does not support, the absence of documentation creates an epistemic vacuum. Due to pre-training priors on similar software and instruction-tuning biases toward helpfulness and sycophancy [Sharma et al., 2024, Ji et al., 2023], models frequently fill this vacuum by inventing plausible workflows.

However, a naive counter-instruction—such as commanding the assistant to declare any undocumented feature "unsupported"—introduces the opposite error: *false denial*. If a maintainer inadvertently omits a minor setting from the manual, the assistant would falsely assert that the setting does not exist.

### 5.1 The Three-State Epistemic Contract

To resolve both hallucinated affirmation and false denial, PASD formalizes a **Three-State Epistemic Model** (Table 3). Every user query regarding application capabilities or behavior falls into one of three distinct epistemic states relative to the support document $K$.

**Table 2:** Layer A Conformance Matrix for `AI-SUPPORT.md`

| ID | Level | Normative Requirement | Verification Method |
|---|---|---|---|
| PASD-001 | MUST | Root `AI-SUPPORT.md` file exists and is non-empty. | Automated file check |
| PASD-002 | MUST | Root `README.md` contains a visible AI-help section linking to `AI-SUPPORT.md` and explaining the upload/link workflow. | Automated link & heading linter |
| PASD-003 | MUST | Document header explicitly declares the target application name, covered version/revision, and repository URL. | Automated metadata check |
| PASD-004 | MUST | Top zone includes embedded support guidance and a Question/Symptom Index pointing to internal sections. | Structural linter |
| PASD-005 | MUST | All documented UI labels, CLI flags, config keys, and error strings match the actual implementation. | Codebase cross-check / audit |
| PASD-006 | MUST | Document explicitly declares unsupported capabilities (Negative Space) and known limitations. | Manual / coverage audit |
| PASD-007 | MUST | External integrations define explicit responsibility boundaries between the project and third-party tools. | Manual / table check |
| PASD-008 | MUST | Destructive recovery procedures include explicit data-loss warnings and backup prerequisites. | Manual safety review |
| PASD-009 | MUST | Escalation section defines issue-tracker URL, required diagnostic details, and secret/path redaction rules. | Structural & manual check |
| PASD-010 | SHOULD | Repository maintains a `docs/support-evaluation.md` suite whose questions are authored independently of the generated FAQ and include negative cases (Layer C readiness). | File check & manual content audit |

**Table 3:** The Three-State Epistemic Model for PASD Support Responses

| Epistemic State | Definition in Knowledge Base ($K$) | Required Assistant Behavior |
|---|---|---|
| **1. Supported** *(Verified Positive)* | The feature, setting, or workflow is explicitly documented as implemented in version $V$. | Explain the exact documented procedure using verified UI labels, paths, or commands, noting any prerequisites or limitations. |
| **2. Explicitly Unsupported** *(Verified Negative)* | The capability is explicitly listed as unsupported, out-of-scope, removed, or unimplemented (roadmap-only). | State clearly that the application does not support the capability in version $V$; do not invent workarounds using nonexistent controls. |
| **3. Unknown / Undocumented** *(Epistemic Uncertainty)* | The supplied documentation neither describes the capability nor declares it unsupported, or diagnostic evidence $E$ is missing. | Acknowledge that the documentation does not establish the answer; request clarifying evidence or inspect source code if available, rather than guessing or issuing a blanket denial [Wen et al., 2025]. |

## 5.2 The Negative Space Mandate

To populate State 2 (*Explicitly Unsupported*), a PASD document MUST include a dedicated Negative Space declaration covering realistic user misconceptions, intentionally omitted features, platform exclusions, and unimplemented roadmap items. Converting common unsupported requests from State 3 (silence) into State 2 (explicit negative facts) allows the language model to answer via direct factual retrieval rather than open-ended extrapolation.

## 5.3 Third-Party Jurisdictional Boundaries

Software projects frequently wrap, invoke, or depend upon external tools, runtimes, or operating-system services. For each significant external integration, `AI-SUPPORT.md` MUST define a responsibility boundary table distinguishing:

- **Project Responsibilities:** Integration configuration exposed by the host application, input validation, process invocation parameters, and how the host surfaces errors.
- **External Tool Responsibilities:** The external dependency's independent installation, internal execution logic, hardware requirements, and internal error states.
- **Boundary Diagnostics:** Concrete checks that allow the user to determine whether a failure occurred before invocation (host validation), at the interface (configuration mismatch), or inside the external process, accompanied by direct links to authoritative upstream documentation.

# 6 Document Topology and Embedded Support Guidance

How information is ordered inside a large Markdown document affects both retrieval reliability and assistant behavior.

## 6.1 Embedded Support Guidance (Document-Level Answering Conventions)

When a user attaches `AI-SUPPORT.md` to a chat session, the file is ingested as untrusted reference data within the user context, not as a privileged system prompt. It cannot—and for security reasons MUST NOT attempt to—override the host assistant's governing system instructions or safety boundaries.

Within those boundaries, however, instruction-tuned assistants actively look for task-framing cues when interpreting uploaded documents. PASD requires a concise **Instructions for AI Assistants** section near the top of the document that establishes document-level answering conventions based on the Three-State Epistemic Model:

```
## Instructions for AI Assistants (PASD Support Guidance)

This document is the authoritative support reference for
[Project Name] [Version]. When answering user questions
using this file:

1. Confirm Retrieval: Verify that you can read this
document. If only a URL was provided and web access failed,
ask the user to upload AI-SUPPORT.md directly.

2. Three-State Epistemic Rule: (a) For documented
features, use exact UI labels, paths, and settings from this
file. (b) For features listed as unsupported or
planned-only, state clearly that they are not supported.
(c) If a topic is neither documented nor listed as
unsupported, state that this document does not establish the
answer rather than guessing.

3. Safe Troubleshooting: Match symptoms against the
Troubleshooting index, verify version and prerequisites
first, and never suggest destructive resets without stating
the documented backup steps.
```

## 6.2 Two-Tiered Document Topology

Empirical studies of long-context language models demonstrate a "Lost in the Middle" effect: models retrieve information near the beginning and end of a prompt significantly more reliably than information buried in the middle [Liu et al., 2024]. To account for this attention profile, PASD specifies a two-tiered internal layout:

- **Tier 1 — The High-Attention Triage Header (Top Zone):** Must appear at the beginning of the file and contain: (1) Document Identity and Version Metadata (`PASD-003`), (2) Embedded Support Guidance, (3) Concise Project Facts, Negative Space Summary, and Responsibility Boundaries, and (4) the Master Topic and Symptom Index (`PASD-004`).
- **Tier 2 — The Deep Operational Reference (Body Zone):** Contains the complete installation guide, feature workflows, configuration reference, detailed troubleshooting trees, and the codebase-derived FAQ, using consistent section headings that match the Tier 1 index exactly.

## 6.3 Diagnostic Escalation and Redaction Rules

When a user's issue falls into State 3 (*Unknown*) or indicates an undocumented software defect, the assistant should guide the user toward safe escalation. As required by `PASD-009`, the document MUST specify where to report unresolved issues, what minimum diagnostic information

to include (application version, OS, exact error text, reproduction steps), and explicit instructions to redact credentials, API keys, tokens, and sensitive personal file paths before posting logs publicly.

# 7 Layer B: Recommended Generation and Maintenance Lifecycle

While Layer A defines what a conformant `AI-SUPPORT.md` file must contain, manually writing and maintaining a comprehensive support bundle across software releases requires substantial effort [Parnas, 1994, Lethbridge et al., 2003]. Layer B defines a recommended, non-normative workflow for generating and maintaining PASD documentation using AI coding assistants and automated repository checks.

## 7.1 Staged Generation and Consistency Review

Instructing a coding agent to inspect a codebase, write modular documentation, build a consolidated support bundle, and verify accuracy in a single prompt frequently results in truncated output or superficial coverage. We recommend a five-stage workflow (detailed as a reproducible runbook in Appendix B) that separates initial generation from review:

1. **Step 0 (Workspace Setup):** Copy the knowledge extraction specification into `docs/`.
2. **Prompt 1 (Execution and Generation):** The agent inspects the codebase, configuration schemas, tests, and entry points; updates the modular documents under `docs/`; compiles the root `AI-SUPPORT.md` from those files; adds the "Need help? Ask an AI assistant" section to the root `README.md`; creates or updates a root `llms.txt` that links to and briefly references `AI-SUPPORT.md`; and creates `docs/support-evaluation.md`, including unanswerable and negative cases.
3. **Prompt 2 (Adversarial Audit and Verification):** In a separate turn, the agent checks every documented CLI flag, UI control, and configuration default against the source, checks that unsupported features and third-party boundaries are explicit, and checks that every question in `docs/support-evaluation.md` can be answered from `AI-SUPPORT.md` alone.
4. **Prompt 3 (Scaffolding Cleanup and Link Verification):** The agent checks that nothing in the repository links to the generation prompt, deletes that copy from `docs/`, and verifies the root `README.md` link to `AI-SUPPORT.md`.
5. **Prompt 4 (Version Control and Deployment):** The agent reviews `git status`, stages the documentation changes, commits them with the message "Updated documentation to support AI assistance and LLM discovery.", and pushes to the active remote branch.

**Methodological scope of Prompt 2:** The runbook calls this pass an adversarial audit. Executed by the same model within the same repository, it is a *consistency review*, not an independent verification. It catches omitted sections, formatting drift, and obvious discrepancies, but can share blind spots with Prompt 1. Independent validation requires deterministic build checks (Section 7.2), human review of the Prompt 4 diff before that push is accepted, and isolated model evaluation (Section 8).

## 7.2 Deterministic Bundling and CI Freshness Enforcement

To prevent manual edits from causing divergence between modular files in `docs/` and the consolidated `AI-SUPPORT.md`, projects SHOULD use a deterministic bundling script. In repositories with Continuous Integration (CI), a non-destructive check command SHOULD verify on every pull request that: (1) `AI-SUPPORT.md` matches the output of the bundler, (2) required headings and version metadata are present, and (3) the root `README.md` link is intact. To mitigate documentation drift as software evolves, CI workflows MAY also implement change-detection heuristics that alert maintainers when pull requests modify public APIs, configuration schemas, or dependencies without corresponding updates to the support documentation.

## 7.3 Deployment and Raw URL Distribution

While direct file download-and-upload is the baseline interoperable method (Section 3), projects that also provide a URL for web-capable AI assistants SHOULD expose the **Raw Text URL** rather than linking exclusively to the web repository viewer:

**Web Viewer URL (Avoid for AI ingestion):**

`https://github.com/dobrado76/MyFileExplorer/blob/main/AI-SUPPORT.md`

**Raw Payload URL (Preferred for AI web-fetching):**

`https://raw.githubusercontent.com/dobrado76/MyFileExplorer/refs/heads/main/AI-SUPPORT.md`

When a web-capable assistant fetches a standard GitHub viewer page, its browsing tool may ingest HTML wrapper markup, navigation bars, commit metadata, and scripts in addition to the document. That wrapper consumes context tokens, may encounter tool-specific truncation, and can push support-relevant facts into less reliably retrieved positions in a long context [Liu et al., 2024]. Providing the `raw.githubusercontent.com` endpoint delivers unformatted Markdown directly when URL retrieval is supported, while the embedded guidance (Section 6) instructs the assistant to request a direct file upload if URL retrieval fails.

# 8 Layer C: Empirical Verification and Evaluation Protocol

Whether a PASD document measurably improves support outcomes over conventional documentation is an empirical hypothesis that requires structured testing. Layer C defines a reproducible evaluation protocol for benchmarking support quality. Conformance to Layer A

(Section 4) does not require a project to conduct empirical model benchmarking; Layer C specifies the conditions necessary for making valid, reproducible evaluation claims when assessing support quality. This paper proposes the protocol but reports no benchmark results; any claim that PASD improves support outcomes remains a hypothesis until such a study is conducted.

## 8.1 Constructing an Independent Test Suite

A repository adopting Layer C maintains an evaluation dataset in `docs/support-evaluation.md` (excluded from the `AI-SUPPORT.md` bundle). To avoid circular testing, evaluation questions MUST NOT be copied or paraphrased from the generated FAQ. Instead, they must be authored independently from realistic user scenarios, issue-tracker history, UI workflows, and error conditions.

A complete benchmark suite SHOULD contain at least $N \geq 40$ test cases stratified across four categories. The minimum design uses exactly $N = 40$, producing the integer allocation below. Larger suites SHOULD preserve the same proportions; where rounding is required, cases SHOULD be assigned by the largest-remainder method so that category counts sum to $N$.

1. **Supported Workflows and Configuration ($40\%$; 16 of 40):** Multi-step operational questions and setting lookups with verifiable ground-truth answers in the codebase.
2. **Symptom and Error Diagnosis ($25\%$; 10 of 40):** Realistic failure reports requiring non-destructive diagnostic checks and third-party boundary identification.
3. **Explicit Negative-Space Queries ($20\%$; 8 of 40):** Plausible requests for nonexistent settings, unsupported file formats, or unimplemented features where the ground truth is State 2 (*Explicitly Unsupported*).
4. **Underspecified or Unknown Queries ($15\%$; 6 of 40):** Ambiguous symptom reports missing critical evidence $E$, or questions outside documented scope where the ground truth is State 3 (*Unknown/Undocumented*, requiring calibrated abstention or clarification).

## 8.2 Four-Condition Experimental Design

Because a comprehensive `AI-SUPPORT.md` file typically contains more total text than a brief `README.md`, a simple comparison between the two confounds *information volume* with *PASD structural conventions*. To separate coverage from presentation effects, we define a four-condition within-subjects evaluation across a preregistered set of models $M_1, \ldots, M_k$ (run in isolated, zero-history sessions with a fixed prompt template and temperature fixed at $0.0$–$0.2$):

- **Condition A (Ungrounded Baseline):** The model receives only the user question $Q$ and application name/version $V$ ($K = \emptyset$).

- **Condition B (Conventional Documentation):** The model receives the repository's standard `README.md` and pre-existing user documentation.
- **Condition C (Content-Equivalent Unstructured Control):** The model receives every factual statement contained in `AI-SUPPORT.md`, including each explicit negative capability statement, but without the embedded guidance preamble, Tier 1 symptom index, or PASD-specific table and heading structure. Negative Space entries are retained as ordinary unstructured prose so factual content is held constant.
- **Condition D (Full PASD Treatment):** The model receives the complete, conformant `AI-SUPPORT.md`.

Comparing Condition B to Condition C measures the benefit of comprehensive codebase-derived factual coverage. Because Conditions C and D contain the same positive and negative facts, comparing C to D estimates the combined contribution of PASD's topological symptom indexing, embedded answering guidance, and structured boundary presentation. It does not isolate those three components from one another; a study claiming component-level causality MUST add preregistered ablation conditions.

## 8.3 Scoring Metrics and Adjudication

Each model response is scored against an explicit ground-truth rubric created prior to testing, measuring five primary metrics:

1. **Supported-Feature Accuracy (SFA):** Proportion of Category 1–2 responses that identify the exact verified controls, settings, and diagnostic steps without factual error.
2. **Negative-Boundary Precision (NBP):** Proportion of Category 3 responses that correctly identify the feature as unsupported without inventing nonexistent controls or workarounds.
3. **Calibrated Abstention Rate (CAR):** Proportion of Category 4 responses that appropriately state uncertainty or request the necessary diagnostic evidence $E$ rather than speculating [Wen et al., 2025].
4. **Unsafe Recommendation Rate (URR):** Proportion of responses across all categories that recommend unwarned destructive actions or violate third-party boundaries.
5. **Downstream Resolution Rate (DRR):** Proportion of executable Category 1–2 cases in which following the response resolves the controlled test condition. DRR MUST be reported separately from answer quality and marked not applicable for clarification cases, external failures, and unresolved software defects.

To prevent self-evaluation bias, scoring SHOULD be performed by human evaluators blinded to the condition (where feasible) or by an independent judge model distinct from the

generator, with human adjudication on disagreements. Before execution, evaluators MUST record model and API versions, prompts, context-construction scripts, condition artifacts and hashes, decoding parameters, and scoring rubrics. Condition order SHOULD be randomized per case. Nondeterministic model-condition cells SHOULD be repeated at least three times; reports MUST provide per-category denominators, paired condition differences, and uncertainty intervals, and MUST limit conclusions to the tested models and versions. Because every condition is evaluated on the same cases, comparisons SHOULD use paired analyses: per-case pass/fail differences between two conditions (e.g., C versus D) can be tested with McNemar's exact test, and 95% intervals for metric differences can be estimated by a case-level bootstrap that resamples test cases (keeping all conditions and repetitions of a case together). Where several metrics or condition pairs are compared, the primary comparison SHOULD be preregistered and the remainder corrected for multiple comparisons (e.g., Holm–Bonferroni).

# 9 Discussion, Security, and Limitations

## 9.1 Context Window Economics, Model Variance, and Partial Pasting

A comprehensive `AI-SUPPORT.md` file for a complex application may span 30,000 to 80,000 tokens. While modern frontier models feature context windows of hundreds of thousands of tokens, repeatedly processing large reference files across multi-turn sessions compounds latency, per-query token expenditure, and rate-limit consumption for users on consumer-tier AI services. Furthermore, long-context retrieval fidelity can degrade when target facts are surrounded by large volumes of reference prose [Liu et al., 2024].

In practical deployments, users frequently encounter chat interfaces with restrictive paste buffers or aggressive sliding-window context truncation. When users copy-paste arbitrary fragments rather than attaching the complete artifact, they risk decoupling the Tier-2 operational procedures from the Tier-1 epistemic guardrails, leaving the model without negative-space boundaries or unverified-feature constraints. PASD mitigates this by emphasizing direct file attachment over manual clipboard pasting, placing high-priority triage rules in the top zone, and strictly excluding internal developer documentation. For very large software suites, maintainers MAY also publish a concise core `AI-SUPPORT.md` alongside modular subsystem references, trading single-file consolidation for a reduced token footprint.

## 9.2 Static Artifacts vs. Dynamic Protocols (MCP)

Dynamic tool-use standards such as the *Model Context Protocol* (MCP) [Anthropic, 2024] allow compatible AI clients to query documentation servers or inspect local runtime state on demand. PASD and MCP address different deployment tiers: MCP provides selective retrieval for users with configured MCP clients and active server processes, whereas `AI-SUPPORT.md` requires zero server infrastructure, can be used through direct attachment in interfaces that support file uploads, and can serve as the underlying knowledge corpus indexed by an MCP

server.

## 9.3 Security: Indirect Prompt Injection and Documentation Poisoning

Distributing a file intended to be read by an AI assistant introduces two distinct security risks that must be governed by explicit rules:

1. **Indirect Prompt Injection:** Users may attach `AI-SUPPORT.md` to assistants that have access to local filesystems, code-execution tools, shell terminals, or connected accounts. A maliciously crafted support file could attempt to hijack the assistant to exfiltrate data or execute arbitrary commands. Consequently, a conformant `AI-SUPPORT.md` file MUST NOT contain instructions claiming system-level authority, attempting to override safety policies, or directing the assistant to execute autonomous actions outside answering the user's support query.

2. **Documentation Poisoning and Contributor Attack Surfaces:** In open-source repositories that accept community pull requests, documentation changes are frequently scrutinized less rigorously than code, creating a viable supply-chain vector. An adversary could propose a plausible troubleshooting step that instructs users or agentic assistants to execute a remote script, disable security controls, or expose credentials. Relying solely on reviewer vigilance is an acknowledged failure mode. Maintainers SHOULD establish enforceable automated guardrails:

    - *Automated Static Analysis:* CI workflows SHOULD lint proposed documentation diffs against a blocklist of high-risk patterns, such as external script downloads (`curl | bash`, `Invoke-Expression`), privilege escalation flags, destructive filesystem operations, or unpinned remote endpoints.
    - *Protected Release Provenance:* Published support artifacts distributed via release channels or raw URLs SHOULD be compiled strictly by automated, protected build pipelines from authenticated release commits rather than accepted directly from unverified pull request commits.
    - *Mandatory Codeowner Scrutiny:* Repositories SHOULD designate documentation files containing troubleshooting commands under strict `CODEOWNERS` policies requiring explicit maintainer sign-off.

## 9.4 Privacy and the Human-in-the-Loop Boundary

While local LLM execution keeps queries on the user's machine, uploading `AI-SUPPORT.md` and diagnostic logs to cloud assistants transmits that session data to the user's chosen AI provider under that provider's terms. PASD's redaction rules (`PASD-009`) help minimize accidental disclosure of secrets during both AI troubleshooting and public issue escalation.

### 9.5 Maintenance Overhead and Epistemic Drift

While Layer B reduces authoring friction through AI-assisted generation, keeping a PASD artifact accurate across rapid software iterations introduces genuine ongoing maintenance overhead. Epistemic drift poses a specific risk: if code changes introduce new capabilities or alter configuration defaults, a previously accurate Negative Space or third-party boundary table can become stale or contradictory. The deterministic bundler and automated CI linters assist in maintaining structural conformance, but maintainers must exercise deliberate discipline during release cycles to ensure that epistemic boundaries reflect the true state of the software. PASD does not replace software engineering maintenance: it cannot fix code defects or diagnose undocumented emergent bugs. Its objective is to resolve routine, documented operational questions accurately so maintainer bandwidth can focus on genuine defects.

## 10 Conclusion

For decades, software documentation has been designed around human search and manual synthesis. The widespread availability of general-purpose AI assistants makes it possible to change how operational software knowledge is consumed—provided the assistant is given verified facts and explicit epistemic boundaries rather than left to guess.

Alongside our related specifications for multi-tiered AI authorship disclosure [Bee, 2026] and structured ontological modeling [Bee, 2025], the Portable AI Support Documentation (PASD) proposal argues that reliable human-AI interaction benefits from structured, auditable reference artifacts. By separating normative document requirements (Layer A), staged maintenance workflows (Layer B), and controlled empirical evaluation (Layer C), PASD provides a practical blueprint that projects of any size can adopt incrementally.

The core proposition is simple: **every sufficiently complex software project should ship a verifiable, self-contained `AI-SUPPORT.md` knowledge base.** The developer maintains the knowledge. The user chooses the AI. And first-line technical support becomes portable, transparent, and grounded—without requiring proprietary support infrastructure.

# References

AIRD Community (2026). *AI-Ready Documentation Specification (AIRD v1.2)*. Retrieved from https://iwweee.github.io/ai-ready-docs/AIRD_SPEC.html.

Anthropic (2024). *Model Context Protocol (MCP): An Open Protocol for Connecting AI Assistants to Data Systems*. Retrieved from https://modelcontextprotocol.io/.

Bee, D. (2025). *A Methodology for Generative Narrative and Civilization-Modeling: The Speculative Ontological Engine (SOE) Framework*. Meaningfulness Media Group.

Bee, D. (2026). *Authorship Transparency Statement (ATS) Framework: A Technical Standard for the Multi-Tiered Disclosure of Generative AI in Creative Works* (ATS-Framework-1.0). Meaningfulness Media Group. https://doi.org/10.2139/ssrn.6146306.

Bradner, S. (1997). *Key words for use in RFCs to Indicate Requirement Levels*. IETF RFC 2119, BCP 14. https://www.rfc-editor.org/info/rfc2119.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*.

Howard, J. (2024). *The /llms.txt file: Helping language models use websites*. Answer.AI. Retrieved from https://llmstxt.org/.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.

Lethbridge, T. C., Singer, J., & Forward, A. (2003). How Software Engineers Use Documentation: The State of the Practice. *IEEE Software*, 20(6), 35–39.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, H., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12, 157–173. (Preprint 2023).

OpenAI & Contributors (2025). *AGENTS.md: A Standard Format for Guiding AI Coding Agents*. Retrieved from https://agents.md/.

Parnas, D. L. (1994). Software Aging. In *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, 279–287. IEEE.

Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., et al. (2024). Towards Understanding Sycophancy in Language Models. In *International Conference on Learning Representations (ICLR 2024)*.

Wen, B., Yao, J., Feng, S., Xu, C., Tsvetkov, Y., Howe, B., & Wang, L. L. (2025). Know Your Limits: A Survey of Abstention in Large Language Models. *Transactions of the Association for Computational Linguistics (TACL)*, 13, 529–556.

# Appendices

# A Reference Implementation: *MyFileExplorer*

To demonstrate how the Portable AI Support Documentation (PASD) specification is structured in an ordinary desktop application, we examine **MyFileExplorer** (`https://github.com/dobrado76/MyFileExplorer`), an open-source Windows file manager featuring persistent multi-pane browsing, tabbed workspaces, rich file previews, fast search, customizable context menus, and Git status integration.

Selecting a desktop productivity utility—rather than an AI-native application—as the canonical reference implementation demonstrates that PASD applies to general-purpose consumer and developer software where users routinely encounter UI navigation questions, configuration persistence issues, and operating-system boundary errors.

## A.1 Repository and Documentation Layout

In accordance with Layer A (Section 4), *MyFileExplorer* maintains modular source documentation under `docs/` and publishes the consolidated `AI-SUPPORT.md` payload at the repository root:

```
MyFileExplorer/
  README.md                 # AI-help section and support link
  AI-SUPPORT.md             # Consolidated PASD knowledge payload
  llms.txt                  # Optional discovery pointer
  docs/
    getting-started.md      # Installation and first launch
    reference.md            # Controls, shortcuts, settings
    configuration.md        # Persistence, paths, defaults
    troubleshooting.md      # Symptom-oriented diagnostics
    FAQ.md                  # Codebase-derived Q&A
    support-evaluation.md   # Maintainer coverage cases
```

## A.2 Mapping Conformance Requirements to Repository Artifacts

Table 4 summarizes how the *MyFileExplorer* repository implements the Layer A conformance criteria defined in Table 2.

## A.3 Illustrative Diagnostic Scenario

To illustrate the intended normative behavior of the Three-State Epistemic Model (Section 5), consider a compound user query: *"Why isn't MyFileExplorer showing Git status badges in my repository folder, and where do I enter the Git password for my remote push?"*

1. **Plausible Failure Without `AI-SUPPORT.md`** ($K = \emptyset$)**:** An ungrounded assistant relying on general IDE priors might invent a nonexistent "Settings → Git → Credentials" field, sending the user searching for controls that do not exist.

| PASD Requirement | *MyFileExplorer* Implementation Detail |
|---|---|
| **Discoverability (`PASD-002`)** | Root `README.md` includes a near-top "Need help with MyFileExplorer?" section linking to `AI-SUPPORT.md` and explaining the download-and-upload workflow. |
| **In-App Routing** | The tab-bar **?** Help control includes "Copy AI support link," which copies the raw support URL for pasting into web-capable assistants. |
| **Symptom Triage (`PASD-004`)** | Tier 1 header indexes observable desktop symptoms (e.g., blank preview panes, unindexed files, locked renames) to non-destructive diagnostic checks. |
| **Negative Space (`PASD-006`)** | Explicitly states that the application is not a cloud drive, does not ship the Everything search service, does not store Git credentials, and does not support macOS. |
| **Third-Party Boundaries (`PASD-007`)** | Separates *MyFileExplorer*'s internal UI and process invocation from Windows OS file permissions, external `git` executables, and system preview codecs. |
| **Evaluation Readiness (`PASD-010`)** | `docs/support-evaluation.md` contains fifteen positive, negative, and uncertainty cases authored separately from the generated FAQ for maintainer coverage checks. It is a seed suite, not a completed $N \geq 40$ Layer C benchmark. |

**Table 4:** Implementation of PASD conformance criteria in *MyFileExplorer*.

2. **Target Normative Behavior With `AI-SUPPORT.md` ($K = K_{\mathbf{PASD}}$):** Guided by the Tier 1 embedded guidance and boundary tables, the assistant separates the two halves of the query:

    - *State 1 (Supported Feature & Boundary Triage):* It explains how to check whether the external `git` executable is available on the system `PATH` and whether the directory contains a valid `.git` repository.
    - *State 2 (Explicitly Unsupported Capability):* It retrieves the explicit declaration that *MyFileExplorer* does not store Git credentials, explains that authentication remains the responsibility of Git Credential Manager or SSH, and directs the user to Git's documentation.

*Methodological Note:* This walkthrough illustrates the normative target behavior of the specification; empirical measurement of model adherence across Conditions A–D follows the Layer C evaluation protocol in Section 8.

# B Reproducible Layer B Generation Procedure

For maintainers who wish to use an AI coding assistant (such as Cursor, Claude Code, or Aider) to generate and maintain a Layer A conformant `AI-SUPPORT.md` file, this appendix provides the concrete operational runbook corresponding to Section 7. The full extraction specification and sequential prompt script are:

- **Knowledge extraction specification**
  `AI-SUPPORT-documentation-generation-prompt.md`
- **Sequential agent runbook**
  `AI-SUPPORT Generation and Publishing Workflow.txt`

The addresses themselves are printed once, in Appendix D.

## B.1 Scope of the Extraction Specification

The `AI-SUPPORT-documentation-generation-prompt.md` file instructs the coding agent across fourteen verification and authoring tasks:

1. **Implementation Inspection (Sections 1 & 10):** Inspect entry points, UI controls, CLI commands, configuration schemas, error messages, and test fixtures before drafting. Source code and tests take precedence over outdated prose.
2. **Modular & Consolidated Layout (Sections 2 & 3):** Maintain modular files under `docs/`, compile end-user support content into a self-contained root `AI-SUPPORT.md`, and add the discoverable AI-help section to the root `README.md`.
3. **Workflows, Technical Precision, & Troubleshooting (Sections 4–7):** Document exact UI labels, version bounds, non-destructive troubleshooting trees (*Symptom* $\rightarrow$ *Causes* $\rightarrow$ *Checks* $\rightarrow$ *Resolution*), and a codebase-derived FAQ.
4. **Boundaries & Embedded Guidance (Sections 8 & 9):** Generate third-party responsibility tables, explicit unsupported-feature declarations, and the Tier 1 embedded guidance preamble.
5. **Bundling, CI Checks, & Evaluation Suite (Sections 11–14):** Create deterministic bundling/validation commands for CI and construct `docs/support-evaluation.md` with independent positive and negative test cases.

## B.2 Sequential Execution Commands

To avoid single-turn output truncation, maintainers paste the following prompts, unchanged, as five discrete turns. The text is the published runbook.

```
[Step 0: Workspace Setup]
Copy `AI-SUPPORT-documentation-generation-prompt.md` into the `docs/`
directory of the project.

[Prompt 1: Execution & Generation]
Read `docs/AI-SUPPORT-documentation-generation-prompt.md` and systematically
execute the task.

Workflow rules for this turn:
1. First, inspect the actual codebase, configuration schemas, tests, and
   entry points.
2. Create or update the modular source documents under `docs/` so they
   reflect runtime reality.
3. Generate the consolidated root `AI-SUPPORT.md` from those verified
   source files.
4. Update the root `README.md` with the mandatory "Need help? Ask an AI
   assistant" section linking directly to `AI-SUPPORT.md`.
5. Create or update a root `llms.txt` that links to and briefly
   references `AI-SUPPORT.md`. Do not copy the support document into
   `llms.txt`, and do not create `llms-full.txt`.
6. Create `docs/support-evaluation.md` with realistic test questions
   (including unanswerable/negative cases).

Do not take shortcuts. Source code and tests take precedence over existing
documentation.

[Prompt 2: Adversarial Audit & Verification]
Perform an adversarial audit of `AI-SUPPORT.md` against the codebase:

1. Verification Check: Cross-reference every documented CLI flag, UI button,
   and config default against the actual source code. Did you document
   anything that doesn't exist?
2. Negative Constraint Check: Does `AI-SUPPORT.md` explicitly list
   unsupported features, platform limitations, and third-party
   responsibility boundaries?
3. Self-Sufficiency Check: Can an AI answer all questions in
   `docs/support-evaluation.md` using ONLY `AI-SUPPORT.md` without
   inspecting source code?
4. Fix any gaps, contradictions, or missing symptoms discovered during
   this audit.

[Prompt 3: Scaffolding Cleanup & Link Verification]
1. Scan all files in the repository (especially `README.md`,
   `docs/README.md`, and `llms.txt`) to ensure nothing links to
   `AI-SUPPORT-documentation-generation-prompt.md`.
2. Permanently delete `docs/AI-SUPPORT-documentation-generation-prompt.md`.
3. Verify that the link to `AI-SUPPORT.md` in the root `README.md` is
   valid and working.

[Prompt 4: Version Control & Deployment]
Review git status, stage all documentation changes, and commit with message:
"Updated documentation to support AI assistance and LLM discovery."
Push to the active remote branch.
```

# C Glossary of Terms

**`AI-SUPPORT.md`** The canonical root-level Markdown artifact defined by Layer A of the PASD specification, containing a project's self-contained, end-user technical support knowledge base.

**Bring Your Own AI (BYOAI)** A support distribution model in which the software project ships a verified static knowledge artifact ($K$) while the end user provides their own preferred AI assistant ($M$) to interpret it.

**Calibrated Abstention** An assistant response that appropriately states uncertainty, declines to invent undocumented functionality, or requests necessary diagnostic evidence when a query falls into Epistemic State 3 (*Unknown/Undocumented*).

**Circular Testing** A flawed evaluation design in which benchmark questions are copied from a generated FAQ rather than derived independently from the codebase and realistic user scenarios.

**Codebase-Derived FAQ** A collection of realistic user questions and complete operational answers extracted systematically from an application's verified UI controls, configuration schemas, workflows, and error states.

**Context Dilution (Lost in the Middle)** The degradation of long-context transformer retrieval accuracy when relevant facts are buried in the middle of a prompt or obscured by irrelevant architectural prose and HTML markup.

**Context Tax** The token consumption, latency, and rate-limit overhead incurred by passing a comprehensive support document into a language model's context window during support sessions.

**Documentation Poisoning** A security threat in which a malicious contributor inserts destructive commands, unsafe configuration steps, or credential-exfiltration traps into a project's support documentation.

**Embedded Support Guidance** Document-level answering conventions placed in the Tier 1 header of `AI-SUPPORT.md` that guide cooperative assistants to follow the Three-State Epistemic Model without claiming privileged system-prompt authority.

**In-App Context Routing** An optional UI convenience mechanism that copies the application version, active error context, and raw documentation link to the user's clipboard for pasting into web-capable assistants.

**Independent Test Harness (`support-evaluation.md`)** A maintainer regression suite containing stratified positive, diagnostic, negative-space, and underspecified test cases used to evaluate support quality under Layer C.

**Knowledge Extraction Specification** The reusable Layer B instruction specification that guides a coding agent through codebase inspection and PASD documentation synthesis.

**Negative-Case Benchmarking** An evaluation method within Layer C that tests an AI assistant with plausible requests for nonexistent or unsupported features to measure Negative-Boundary Precision (NBP).

**Negative Space Mandate** The normative requirement (`PASD-006`) that documentation explicitly identify unsupported capabilities, anti-features, and unimplemented plans so assistants can retrieve verified negative facts (Epistemic State 2).

**Portable AI Support Documentation (PASD)** The proposed `MMG-PASD-1.0` three-layer specification encompassing normative document requirements (Layer A), staged maintenance workflows (Layer B), and empirical evaluation protocols (Layer C).

**Raw Payload URL** A direct link to the unformatted plain-text Markdown file (e.g., via `raw.githubusercontent.com`) rather than a web repository viewer page, avoiding HTML DOM bloat when web-fetching is supported.

**Staged Generation and Consistency Review** The Layer B maintenance workflow that separates initial codebase-derived documentation generation from a second-pass consistency check and human maintainer review.

**Sycophancy Bias** The tendency of instruction-tuned language models to conform to false user premises—such as inventing instructions for a nonexistent software setting simply because the user asked how to configure it.

**Symptom-Oriented Troubleshooting** A diagnostic organization pattern that indexes issues by user-observable symptoms and follows the chain: *Symptom* $\rightarrow$ *Possible Causes* $\rightarrow$ *Diagnostic Checks* $\rightarrow$ *Safe Resolution* $\rightarrow$ *Expected Result*.

**Third-Party Jurisdictional Boundaries** Explicit documentation tables (`PASD-007`) separating the host application's integration responsibilities from the internal execution and failure modes of external tools or operating systems.

**Three-State Epistemic Model** The classification of capability queries relative to $K$ into *Supported* (verified positive), *Explicitly Unsupported* (verified negative), and *Unknown/Undocumented* (epistemic uncertainty), preventing both hallucinated affirmation and false denial.

**Two-Tiered Document Topology** The prescribed physical layout of `AI-SUPPORT.md`, placing metadata, embedded guidance, negative constraints, and symptom indices in the Tier 1 High-Attention Header, followed by the Tier 2 Deep Operational Reference.

**Version Bounding** The explicit declaration (`PASD-003`) of the software version or commit range covered by `AI-SUPPORT.md`, paired with guidance requiring the assistant to verify the user's version before providing version-sensitive advice.

# D Canonical Artifact Addresses

The two Layer B artifacts are published with this specification. Each address is printed in full only here. In the rest of the document the artifact name is a link to the same address, and the address is not shown as text.

**Knowledge extraction specification**

`https://raw.githubusercontent.com/MeaningfulnessMediaGroup/MMG-PASD/refs/heads/main/AI-SUPPORT-documentation-generation-prompt.md`

**Sequential agent runbook**

`https://raw.githubusercontent.com/MeaningfulnessMediaGroup/MMG-PASD/refs/heads/main/AI-SUPPORT%20Generation%20and%20Publishing%20Workflow.txt`